



Effective Modern C++

CH 01: Deducing Types

▼ Template type deduction

```
template<typename T>
void f(ParamType param);

f(expr); // call f with some expression
```

The type deduced for T is dependent not just on the type of expr, but also on the form of ParamType. There are three cases:

- **ParamType is a pointer or reference type, but not a universal reference:**

- If expr's type is a reference or pointer, ignore the reference or pointer part .
- Then pattern-match expr's type against ParamType to determine T.

- **ParamType is a universal reference:**

- If expr is an lvalue, both T and ParamType are deduced to be lvalue references.

That's doubly unusual. First, it's the only situation in template type

deduction where T is deduced to be a reference. Second, although ParamType is declared using the syntax for an rvalue reference, its deduced type is an lvalue reference.

- If expr is an rvalue, the “normal” (i.e., Case 1) rules apply.
- **ParamType is neither a pointer nor a reference:**
 - As before, if expr’s type is a reference, ignore the reference part.
 - If, after ignoring expr’s reference-ness, expr is const, ignore that, too. If it’s volatile, also ignore that.

Array and Function Arguments

During template type deduction, arguments that are array or function names decay to pointers, unless they’re used to initialize references.

```
void someFunc(int, double); // someFunc is a function;
//
type is void(int, double)

template<typename T>
void f1(T param);           // in f1, param passed by value
template<typename T>
void f2(T& param);          // in f2, param passed by ref

f1(someFunc);               // param deduced as ptr-to-func;
// type is void (*)(int, double)

f2(someFunc);               // param deduced as ref-to-func;
// type is void (&)(int, double)
```

▼ Understand auto type deduction

When a variable is declared using `auto`, `auto` plays the role of `T` in the template, and the type specifier for the variable acts as `ParamType`.

Array and function names undergo pointer decay for non-reference type specifiers, a behavior that also occurs during `auto` type deduction.

```
const char name[] = "R. N. Briggs";    // name's type is
const char[13]
auto arr1 = name;                      // arr1's type is c
const char*
auto& arr2 = name;                     // arr2's type is co
const char (&)[13]
```

The treatment of braced initializers is the only way in which `auto` type deduction and template type deduction differ. When an `auto`-declared variable is initialized with a braced initializer, the deduced type is an instantiation of `std::initializer_list`. But if the corresponding template is passed the same initializer, type deduction fails, and the code is rejected. So the only real difference between `auto` and template type deduction is that `auto` assumes that a braced initializer represents a `std::initializer_list`, but template type deduction doesn't.

`auto` in a function return type or a lambda parameter implies template type deduction, not `auto` type deduction.

▼ Understand decltype

`decltype` typically parrots back the exact type of the name or expression.

```
bool f(const Widget& w); // decltype(w) is const Widget&
                          // dec
decltype(f) is bool(const Widget&)
```

In C++11, perhaps the primary use for `decltype` is declaring function templates where the function's return type depends on its parameter types.

```
// In C++ 11
template<typename Container, typename Index> // works, but
requires refinement
auto authAndAccess(Container& c, Index i) -> decltype(c
[i])
{
    authenticateUser();
    return c[i];
}
```

/*
C++11 permits return types for single-statement lambdas to be deduced, and C++14 extends this to both all lambdas and all functions, including those with multiple statements. In the case of authAndAccess, that means that in C++14 we can omit the trailing return type, leaving just the leading auto.
*/

```
// In C++ 14 (wrong)
template<typename Container, typename Index>
auto authAndAccess(Container& c, Index i)
{
    authenticateUser();
    return c[i]; // return type deduced from c[i]
}
```

/*
Here, d[5] returns an int&, but auto return type deduction for authAndAccess will strip off the reference, thus yielding a return type of int. That int, being the return value of a function, is an rvalue, and the code above thus attempts to assign 10 to an

```
rvalue int. That's forbidden in C++, so the code won't compile.
```

```
*/
```

```
/*
```

```
To get authAndAccess to work as we'd like, we need to use  
decltype type deduction  
for its return type, i.e., to specify that authAndAccess should  
return exactly the same  
type that the expression c[i] returns. The guardians of C++,  
anticipating the need to  
use decltype type deduction rules in some cases where types  
are inferred, make this  
possible in C++14 through the decltype(auto) specifier.
```

```
*/
```

```
// In C++ 14 (correct)
```

```
template<typename Container, typename Index>  
decltype(auto) authAndAccess(Container& c, Index i)  
{  
    authenticateUser();  
    return c[i];  
}
```

The use of `decltype(auto)` is not limited to function return types. It can also be convenient for declaring variables when you want to apply the `decltype` type deduction rules to the initializing expression:

```
Widget w;  
const Widget& cw = w;  
auto myWidget1 = cw; // auto type deduction: myWidget1's type  
is Widget  
decltype(auto) myWidget2 = cw; // decltype type deduction:  
const Widget&
```

Wrapping the name `x` in parentheses—`(x)`—yields an expression more complicated than a name. Being a name, `x` is an lvalue, and C++ defines the expression `(x)` to be an lvalue, too. `decltype((x))` is therefore `int&`. Putting parentheses around a name can change the type that `decltype` reports for it!

```
decltype(auto) f1()
{
    int x = 0;
    ...
    return x; // decltype(x) is int, so f1 returns int
}

decltype(auto) f2()
{
    int x = 0;
    ...
    return (x); // decltype((x)) is int&, so f2 returns in
               // t&
}
```

▼ Know how to view deduced types

```
#include <boost/type_index.hpp>
template<typename T>
void f(const T& param)
{
    using std::cout;
    using boost::typeindex::type_id_with_cvr;
    // show T
    cout << "T = "
    << type_id_with_cvr<T>().pretty_name()
    << '\n';
    // show param's type
}
```

```

cout << "param = "
<< type_id_with_cvr<decltype(param)>().pretty_name()
<< '\n';
...
}

```

The way this works is that the function template `boost::typeid::type_id_with_cvr` takes a type argument (the type about which we want information) and doesn't remove `const`, `volatile`, or reference qualifiers (hence the "with_cvr" in the template name).

The result is a `boost::typeid::type_index` object, whose `pretty_name` member function produces a `std::string` containing a human-friendly representation of the type.

CH 02: auto

▼ Prefer auto to explicit type declarations

`auto` / `std::function` object can represent types known only to compilers as shown below:

```

// comparison func for Widgets pointed to by std::unique_ptr
// using auto
auto derefUPLess = [](const std::unique_ptr<Widget>& p1,
const std::unique_ptr<Widget>& p2) { return *p1 < *p2; };

```

```

/*
std::function is a template in the C++11 Standard Library
that generalizes the idea
of a function pointer. Whereas function pointers can point
only to functions, however, std::function objects can refer to any callable object,
i.e., to anything that can
be invoked like a function. Just as you must specify the t

```

```

ype of function to point to
when you create a function pointer (i.e., the signature of
the functions you want to
point to), you must specify the type of function to refer
to when you create a
std::function object. You do that through std::function's
template parameter.
*/

```

```

// comparison func for Widgets pointed to by std::unique_p
trs using function object
std::function<bool(const std::unique_ptr<Widget>&,
const std::unique_ptr<Widget>&)>
derefUPLess = [](const std::unique_ptr<Widget>& p1, const
std::unique_ptr<Widget>& p2) { return *p1 < *p2; };

```

An auto-declared variable holding a closure has the same type as the closure, and as such

it uses only as much memory as the closure requires. The type of a `std::function` declared variable holding a closure is an instantiation of the `std::function` template, and that has a fixed size for any given signature. This size may not be adequate for the closure it's asked to store, and when that's the case, the `std::function` constructor will allocate heap memory to store the closure.

The result is that the `std::function` object typically uses more memory than the auto-declared object. And, thanks to implementation details that restrict inlining and yield indirect function calls, invoking a closure via a `std::function` object is almost certain to be slower than calling it via an auto-declared object.

`std::vector<int>::size_type` is an unsigned type whose size depends on the platform, but `unsigned` is always 32-bit on Windows. So code using `unsigned` instead of `size_type` may work on 32-bit Windows but break on 64-bit Windows due to size mismatches. Using the proper `size_type` avoids these portability bugs.

```
auto sz = v.size(); // sz's type is std::vector<int>::size_type
```

▼ Use the explicitly typed initialiser idiom when auto deduces undesired types

```
bool highPriority = features(w)[5]; // declare highPriority's type explicitly

auto highPriority = features(w)[5]; // deduce highPriority's type
```

`std::vector<bool>` doesn't return a real `bool&` from `operator[]`; it returns a proxy object `std::vector<bool>::reference` because its bits are packed and C++ can't reference individual bits. When you write `bool highPriority = features(w)[5];`, this proxy converts to a real `bool`, so everything is fine.

`std::vector<bool>::reference` is an example of a proxy class: a class that exists for the purpose of emulating and augmenting the behaviour of some other type. Proxy classes are employed for a variety of purposes.

`std::vector<bool>::reference` exists to offer the illusion that `operator[]` for `std::vector<bool>` returns a reference to a bit.

But when you write `auto highPriority = features(w)[5];`, `highPriority` becomes that proxy type itself—not a `bool`. Since `features(w)` returns a temporary `std::vector<bool>`, the proxy ends up storing a pointer into that temporary's internal memory. Once the temporary is destroyed at the end of the statement, the proxy in `highPriority` contains a dangling pointer, and using it (e.g., passing it to `processWidget`) triggers undefined behaviour.

Sometimes `auto` deduces type incorrectly and when that's the case it is important to guide `auto` to the right type instead of falling back to manual type declarations. The solution is to force a different type deduction. The way you do that is what I call the explicitly typed initialiser idiom.

The explicitly typed initialiser idiom involves declaring a variable with `auto`, but casting the initialisation expression to the type you want `auto` to deduce.

Here's how

it can be used to force `highPriority` to be a `bool`, for example:

```
auto highPriority = static_cast<bool>(features(w)[5]);
```

Here, `features(w)[5]` continues to return a `std::vector<bool>::reference` object, just as it always has, but the cast changes the type of the expression to `bool`, which `auto` then deduces as the type for `highPriority`.

CH 03: Moving to Modern C++

▼ Distinguish between `()` and `{}` when creating objects

The “confusing mess” lobby points out that the use of an equals sign for initialisation often misleads C++ newbies into thinking that an assignment is taking place, even though it's not.

```
Widget w1; // call default constructor
Widget w2 = w1; // not an assignment; calls copy ctor
w1 = w2; // an assignment; calls copy operator=
```

To address the confusion of multiple initialization syntaxes, as well as the fact that they don't cover all initialization scenarios, C++11 introduces uniform initialization: a single initialization syntax that can, at least in concept, be used anywhere and express everything.

```
std::vector<int> v{ 1, 3, 5 }; // v's initial content is
1, 3, 5
```

Braces can also be used to specify default initialization values for non-static data members. This capability—new to C++11—is shared with the `=` initialization syntax, but not with parentheses:

```
class Widget {
...
}
```

```
private:
int x{ 0 }; // fine, x's default value is 0
int y = 0; // also fine
int z(0); // error!
};
```

On the other hand, uncopyable objects (e.g., `std::atomic`s) may be initialized using braces or parentheses, but not using `"="`:

```
std::atomic<int> ai1{ 0 }; // fine
std::atomic<int> ai2(0); // fine
std::atomic<int> ai3 = 0; // error!
```

A novel feature of braced initialization is that it prohibits implicit narrowing conversions among built-in types. If the value of an expression in a braced initializer isn't guaranteed to be expressible by the type of the object being initialized, the code won't compile:

```
double x, y, z;
...
int sum1{ x + y + z }; // error! sum of doubles may
// not be expressible as int

/*
NOTE: Initialization using parentheses and "=" doesn't check
for narrowing conversions,
because that could break too much legacy code:
*/

int sum2(x + y + z); // okay (value of expression truncated
to an int)
int sum3 = x + y + z; // ditto
```

Another noteworthy characteristic of braced initialization is its immunity to C++'s most vexing parse. A side effect of C++'s rule that anything that can be

parsed as a declaration must be interpreted as one, the most vexing parse most frequently afflicts developers when they want to default-construct an object, but inadvertently end up declaring a function instead.

```
/*
The root of the problem is that if you want to call a con-
structor with an argument, you can do it like this:
*/

Widget w1(10); // call Widget ctor with argument 10

/*
but if you try to call a Widget constructor with zero argu-
ments using the analogous
syntax, you declare a function instead of an object:
*/

Widget w2(); // most vexing parse! declares a function
              // named w2 that returns a Widget!

/*
Functions can't be declared using braces for the parameter
list, so default-
constructing an object using braces doesn't have this prob-
lem:
*/

Widget w3{}; // calls Widget ctor with no args
```

Drawback of Braced Initializer: When a class has a `std::initializer_list` constructor, **brace initialization always prefers it**, even if the match is worse than other overloads.

So `Widget w2{10, true}` and `Widget w4{10, 5.0}` will call the `initializer_list<long double>` constructor, **not** the more natural `(int, bool)` or `(int, double)`

constructors.

```
class Widget {
public:
    Widget(int i, bool b);
    Widget(int i, double d);
    Widget(std::initializer_list<long double> il);
    ...
};

Widget w1(10, true); // uses parens and, as before,
                      // calls first ct
or

Widget w2{10, true}; // uses braces, but now calls
                      // std::initializ
er_list ctor
                      // (10 and true c
onvert to long double)

Widget w3(10, 5.0); // uses parens and, as before,
                     // calls second ct
or

Widget w4{10, 5.0}; // uses braces, but now calls
                     // std::initialize
r_list ctor
                     // (10 and 5.0 con
vert to long double)
```

Brace initialization prefers the `std::initializer_list` constructor over copy/move constructors, even if it requires converting a `Widget` to `float` and then to `long double`. So `{w4}` triggers the `initializer_list` constructor, while `(w4)` uses normal copy/move behaviour.

```

class Widget {
public:
    Widget(int, bool);
    Widget(int, double);
    Widget(std::initializer_list<long double>);
    operator float() const;
};

Widget w5(w4);           // copy ctor
Widget w6{w4};           // initializer_list ctor (w4 → fl
                          oat → long double)

Widget w7(std::move(w4)); // move ctor
Widget w8{std::move(w4)}; // initializer_list ctor again

```

Empty braces `{}` mean *no arguments*, so a class with both a default constructor and an `initializer_list` constructor will use the **default constructor**. To call the `initializer_list` constructor with an empty list, you must pass `{}` *as an argument* using extra braces or parentheses.

```

class Widget {
public:
    Widget();           // default ctor
    Widget(std::initializer_list<int>); // init_list ctor
};

Widget w1;           // default ctor
Widget w2{};         // default ctor
Widget w3();         // function declaration (most vexing parse)

Widget w4({});       // initializer_list ctor (empty list)
Widget w5{{{}}};     // initializer_list ctor (empty list)

```

Adding a `std::initializer_list` constructor drastically changes overload resolution: **brace initialization will strongly prefer the `initializer_list` overload**, often ignoring other constructors. This means adding such a constructor later can silently break existing client code. So if you design a class, make sure parentheses `{}` and braces `()` lead to the same behavior, or avoid `initializer_list` overloads unless truly necessary.

```
class Widget {
public:
    Widget(int, int);           // old ctor
    // Adding this later can hijack calls using {...}
    Widget(std::initializer_list<int>); // new ctor
};

Widget w1(10, 20); // calls (int,int)
Widget w2{10, 20}; // now calls initializer_list<int> in
                    // stead!
```

▼ Prefer `nullptr` to 0 and `NULL`

`nullptr` has the special type `std::nullptr_t`, which implicitly converts to **any pointer type** but **not** to integers. That's why `nullptr` behaves like a "pointer of all types" and avoids the ambiguity caused by using `0` or `NULL`. So in overload resolution, `nullptr` always picks a pointer overload.

```
void f(int);
void f(void*);

f(nullptr); // calls f(void*) – nullptr is not integral
```

Using `nullptr` instead of `0` or `NULL` avoids confusion in overload resolution **and** makes code clearer. With `auto`, checking against `0` doesn't reveal whether the variable is a pointer or an integer, but comparing against `nullptr` makes it obvious the variable is a pointer.

```

auto result = findRecord( /* arguments */ );
if (result == 0) {
    // Does findRecord returns ptr or integer type (not clear)
}

auto result = findRecord( /* arguments */ );
if (result == nullptr) {
    // findRecord must be returning ptr type
}

```

Using `0` or `NULL` with templates causes type-deduction failures because templates deduce their **real integral type** (int), not “null pointer.” This leads to invalid calls when the function expects a smart pointer or raw pointer. `nullptr` avoids this because it has type `std::nullptr_t`, which converts to **any pointer type**. This makes template code behave correctly and avoids overload resolution surprises.

```

int f1(std::shared_ptr<Widget>);
double f2(std::unique_ptr<Widget>);
bool f3(Widget*);

template<typename Func, typename Mux, typename Ptr>
decltype(auto) lockAndCall(Func func, Mux& m, Ptr p) {
    std::lock_guard<Mux> g(m);
    return func(p);
}

lockAndCall(f1, f1m, 0);           // ❌ deduces Ptr = int → type error
lockAndCall(f2, f2m, NULL);       // ❌ deduces Ptr = int-like → type error
lockAndCall(f3, f3m, nullptr);    // ✓ Ptr = std::nullptr_t → converts to Widget*

```

▼ Prefer alias declarations to typedefs

C++11 introduced **alias templates** (`using`) which are more powerful and cleaner than `typedef` , especially when templates are involved.

1. Alias templates can be templated

Typedefs cannot be alias declarations may be templated (in which case they're called alias templates), while typedefs cannot. This gives C++11 programmers a straightforward mechanism for expressing things that in C++98 had to be hacked together with typedefs nested inside templated structs.

```
template<typename T>
using MyAllocList = std::list<T, MyAlloc<T>>;

MyAllocList<Widget> lw;    // Using alias declaration

/*****
*****/

template<typename T>
struct MyAllocList {
    typedef std::list<T, MyAlloc<T>> type;
};

MyAllocList<Widget>::type lw;    // Using typedefs
```

2. Using typedefs inside templates requires `typename`

When using a typedef that depends on a template parameter (a *dependent type*), the compiler forces you to write `typename` . This is required because compilers *cannot be sure* the name refers to a type.

```
template<typename T>
class Widget {
```

```
private:
    typename MyAllocList<T>::type list; // must use 'typename'
};
```

3. Alias templates **remove the need for** `typename` **and** `::type`

When the same code uses an alias template:

```
template<typename T>
using MyAllocList = std::list<T, MyAlloc<T>>;

template<typename T>
class Widget {
private:
    MyAllocList<T> list; // no 'typename', no '::type'
};
```

Why?

- The compiler *already knows* that an alias template always names a **type**.
- So `MyAllocList<T>` is **not considered dependent**, removing the `typename` requirement.

4. Why compilers treat the typedef version differently

When compilers process the Widget template and encounter the use of `MyAllocList<T>` (i.e., use of the alias template), they know that `MyAllocList<T>` is the name of a type, because `MyAllocList` is an alias template: it must name a type. `MyAllocList<T>` is thus a non-dependent type, and a `typename` specifier is neither required nor permitted.

On the other hand, when compilers see `MyAllocList<T>::type` (i.e., use of the nested typedef) in the Widget template, on the other hand, they can't know for sure that it names a type, because there might be a specialization of `MyAllocList` that they haven't yet seen where `MyAllocList<T>::type`

refers to something other than a type. That sounds crazy, but don't blame compilers for this possibility. It's the humans who have been known to produce such code.



Example: Why `typename` Is Needed

The compiler cannot assume that `MyAllocList<T>::type` is always a type.

Someone might create a specialization where `type` is **not** a type, which forces the compiler to be cautious.

```
// NORMAL CASE
template<typename T>
struct MyAllocList {
    typedef std::list<T> type;    // 'type' is a type
};

MyAllocList<int>::type x;    // OK: x is std::list<int>

// NOTE: A specialization can change the meaning

template<>
struct MyAllocList<int> {
    static int type;    // now 'type' is NOT a type!
};

MyAllocList<int>::type x;    // ERROR: 'type' is a variable

// Hence, when inside the template:

template<typename T>
class Widget {
    typename MyAllocList<T>::type list;    // 'typename'
```

```
required  
};
```

5. C++11 introduced **type traits**, a collection of templates in `<type_traits>` that let you transform types (remove const, strip references, add references, etc.).

Each transformation follows the same form: `std::transformation<T>::type`

```
// C++ 11 Examples  
  
std::remove_const<T>::type           // turns 'const T'  
into 'T'  
std::remove_reference<T>::type       // turns 'T&' or 'T  
&&' into 'T'  
std::add_lvalue_reference<T>::type   // turns 'T' into  
'T&'
```

These work, but they are verbose. You must always write `::type`, and inside templates you must also write `typename`.

The reason is simple: C++11 type traits are implemented as **nested typedefs inside structs**, an older technique that is more awkward compared to alias templates.

C++14 improves this with alias templates: The standards committee realized alias templates are cleaner, so in C++14 they added a shorter version for every type trait: `std::transformation_t<T>`.

```
// C++ 14 Examples  
  
std::remove_const_t<T>  
std::remove_reference_t<T>  
std::add_lvalue_reference_t<T>
```

If you're using C++11 only, you can create your own `_t` aliases easily: This removes the need to repeatedly write `::type` and `typename`.

```

template <class T>
using remove_const_t = typename std::remove_const<T>::type;

template <class T>
using remove_reference_t = typename std::remove_reference<T>::type;

template <class T>
using add_lvalue_reference_t =
                                typename
std::add_lvalue_reference<T>::type;

```

▼ Prefer scoped enums to unscoped enums

1. Unscoped Enums (C++98 style)

- Declared using `enum`
- Enumerator names leak into the surrounding scope
- Can cause name conflicts

```

enum Color { black, white, red }; // names enter outer scope

auto white = false; // ERROR: 'white' already exists

```

Issue: black, white and red become regular names in the enclosing scope, so new variables with the same names are not allowed.

- Enumerators for unscoped enums implicitly convert to integral types (and, from there, to floating-point types).

```

enum Color { black, white, red }; // unscoped enum
std::vector<std::size_t> // func. returning
primeFactors(std::size_t x); // prime factors of x
Color c = red;

```

```
...
if (c < 14.5) {
    auto factors = primeFactors(c); // compare Color to
double (!)

// compute prime factors

// of a Color (!)
...
}
```

- **Forward Declaration (C++11 improvement):**

- C++98 did **not** allow forward-declaring unscoped enums.
- C++11 allows it **only if you specify an underlying type**.

```
// C++98
enum Color; // error!

// C++11
enum Color : std::uint8_t; // OK – fixed underlying type
```

- At least one situation where unscoped enums may be useful. That's when referring to fields within C++11's `std::tuples`. For example, suppose we have a tuple holding values for the name, email address, and reputation value for a user at a social networking website:

```
enum UserInfoFields { uiName, uiEmail, uiReputation };

UserInfo uInfo;

auto val = std::get<uiEmail>(uInfo); // ah, get value of
email field
// clearer than std::get<1>
```

```

/*
NOTE: What makes this work is the implicit conversion f
rom UserInfoFields to
std::size_t, which is the type that std::get requires.
*/

```

2. Scoped Enums (C++11 – enum class)

- Declared using `enum class`
- Enumerator names stay inside the enum's scope
- Prevents naming conflicts

```

enum class Color { black, white, red }; // names are scop
ed

auto white = false; // OK (no conflict)

Color c = white; // ERROR: 'white' not in this sco
pe
Color c = Color::white; // Correct
auto c = Color::white; // Preferred

```

- Scoped enumerators are much more strongly typed i.e., no implicit conversion from enumerators in a scoped enum to any other type.

```

enum class Color { black, white, red }; // enum is now sco
ped
Color c = Color::red; // as before, but with scope qualifi
er
...
if (c < 14.5) { // error! can't comp
are

// Color and double

```

```

    auto factors = primeFactors(c); // error! can't pass
    Color to
                                     // function expectin
    g std::size_t
    ...
    }

```

- However, if you really want to convert types from Color to a different type, do what you always do to twist the type system to your wanton desires—use a cast:

```

if (static_cast<double>(c) < 14.5) { // odd code, but it's
    valid
    auto factors = primeFactors(satic_cast<std::size_t>(c));
    // suspect, but it compiles
    ...
}

```

- Scoped enums **can always be forward-declared** because their underlying type is known (`int` by default).

```

enum class Status; // OK
void process(Status s);

```

- You may specify the underlying type:

```

enum class Status : std::uint32_t; // Underlying type is not
std::uint32_t

// NOTE: Underlying type can be declared on definitions as
well.

enum class Status : std::uint32_t {
    good = 0,

```

```

    failed = 1,
    incomplete = 100,
    corrupt = 200,
    audited = 500,
    indeterminate = 0xFFFFFFFF
};

```

- If you want scoped enums to behave like unscoped enums when indexing tuple fields, you can use the following approach:

```

// C++11 example:
template<typename E>
constexpr typename std::underlying_type<E>::type toUType(E
e) noexcept {
    return static_cast<typename std::underlying_type<E>::t
ype>(e);
}

// C++14 example:
template<typename E>
constexpr auto toUType(E enumerator) noexcept {
    return static_cast<std::underlying_type_t<E>>(enumerat
or);
}

// Usage:
auto val = std::get<toUType(UserInfoFields::uiEmail)>(uInf
o);

```

▼ Prefer deleted functions to private undefined ones

Sometimes you want to **prevent other developers** from calling certain functions—especially the **special member functions** that C++ may auto-generate:

- Copy constructor

- Copy assignment operator
- (others exist, but these two are the most commonly suppressed)

In C++98, preventing copying required a workaround.

C++11 provides a clean and explicit solution using `= delete`

In C++98, you block copying by:

- Declaring the copy constructor and copy assignment operator **private**
- **Not defining** them

This causes:

- Compile-time prevention for client code
- Linker errors if internal code (friends/members) tries to use them

```
template <class charT, class traits = char_traits<charT>>
class basic_ios : public ios_base {
public:
    ...
private:
    basic_ios(const basic_ios&);           // declared, but not defined
    basic_ios& operator=(const basic_ios&); // declared, but not defined
};
```

Problems with this approach:

- Errors may appear **only at link time**
- Compilers may produce confusing error messages

C++11 introduces **deleted functions**, allowing you to explicitly disable special member functions:

```
template <class charT, class traits = char_traits<charT>>
class basic_ios : public ios_base {
public:
```

```

    ...
    basic_ios(const basic_ios&) = delete;           // cannot copy
    basic_ios& operator=(const basic_ios&) = delete; // cannot assign
};

```

Advantages of using =delete functions:

- Deleted functions cannot be used **anywhere**, even by friends or member functions.
- Errors occur at **compile time**, not link time.
- Clearer intent: "This function must not be used."

Why deleted functions are public:

- The compiler checks **access** before checking for deleted status.
- If they were private, some compilers might show only *"function is private"* — misleading.
- Declaring them public produces clearer messages: *"use of deleted function ..."*

Another advantage of using `= delete` is that it works **for non-member functions (unlike private functions)**.

```

bool isLucky(int number);           // original function

bool isLucky(char) = delete;        // reject chars
bool isLucky(bool) = delete;        // reject bools
bool isLucky(double) = delete;      // reject doubles (and floats)

// C++ allows implicit conversions:

if (isLucky('a')) // is 'a' a lucky number?

```

```

if (isLucky(true)) // is "true"?
if (isLucky(3.5)) // should we truncate to 3 before checking for luckiness?

// Result

if (isLucky('a')) // ERROR: call to deleted function
if (isLucky(true)) // ERROR
if (isLucky(3.5f)) // ERROR (float → double → deleted)

// NOTE: Deleted functions still participate in overload resolution, so invalid calls are caught at compile time.

```

The biggest advantage of using `= delete` is that you can disable certain template usages explicitly. For example, consider the following code:

```

template<typename T>
void processPointer(T* ptr);

```

There are two special cases in the world of pointers. One is `void*` pointers, because there is no way to dereference them, to increment or decrement them, etc. The other is `char*` pointers, because they typically represent pointers to C-style strings, not pointers to individual characters. These special cases often call for special handling, and, in the case of the `processPointer` template, let's assume the proper handling is to reject calls using those types. That is, it should not be possible to call `processPointer` with `void*` or `char*` pointers. That's easily enforced. Just `= delete` those instantiations:

```

template<>
void processPointer<void>(void*) = delete;
template<>
void processPointer<char>(char*) = delete;

```

Interestingly, if you have a function template inside a class, and you'd like to disable some instantiations by declaring them private (à la classic C++98

convention), you can't, because it's not possible to give a member function template specialization a different access level from that of the main template.

If `processPointer` were a member function template inside `Widget`, for example, and you wanted to disable calls for `void*` pointers, this would be the C++98 approach, though it would not compile:

```
class Widget {
public:
    ...
    template<typename T>
    void processPointer(T* ptr)
    { ... }
private:
    template<> // error!
    void processPointer<void>(void*);
};
```

The problem is that template specializations must be written at namespace scope, not class scope. This issue doesn't arise for deleted functions, because they don't need a different access level. They can be deleted outside the class (hence at namespace scope):

```
class Widget {
public:
    ...
    template<typename T>
    void processPointer(T* ptr)
    { ... }
    ...
};

template<>
void Widget::processPointer<void>(void*) = delete; // still public, but deleted
```

▼ Declare overriding functions override

Overriding vs Overloading:

- **Overriding:** Derived class provides a new implementation of a **virtual** base function.
- **Overloading:** Same function name but different parameters — *completely unrelated*.

```
class Base {
public:
    virtual void doWork();
};

class Derived : public Base {
public:
    void doWork(); // overrides Base::doWork
};

std::unique_ptr<Base> p = std::make_unique<Derived>();
p->doWork(); // calls Derived::doWork
```

A derived function overrides a base virtual only if ALL of these match:

- Function in base must be `virtual`
- Same function **name**
- Same **parameter types**
- Same **constness**
- Compatible **return type**
- Compatible **exception specification**
- C++ 11 added a new rule that the functions must have the **same reference qualifier** (`&` or `&&`).

```
void f() &;    // only for lvalues
void f() &&;   // only for rvalues
```

Overriding Errors Are Easy to Miss:

```
// This code compiles but nothing overrides:
class Base {
public:
    virtual void mf1() const;
    virtual void mf2(int);
    virtual void mf3() &;
    void mf4() const;    // not virtual
};

class Derived : public Base {
public:
    void mf1();           // missing const
    void mf2(unsigned int); // different parameter type
    void mf3() &&;        // different ref qualifier
    void mf4() const;     // base not virtual
};

// SOLUTION: Use override always

class Derived : public Base {
public:
    void mf1() const override;
    void mf2(int) override;
    void mf3() & override;
    void mf4() const override;
};
```

```
// NOTE: If anything is wrong → compiler error.  
// This is exactly what you want.
```

Benefits of Always Using `override`

- Ensures your function truly overrides
 - If not, compiler errors immediately.
- Helps with refactoring
 - If you change a base class virtual signature:
 - all broken overrides in derived classes will fail to compile
 - easy to find all impacted code
- Prevents silent bugs
 - Without `override`, incorrect signatures compile fine but behave wrong.

C++ has always had keywords, but C++11 introduces two contextual keywords, `override` and `final`. These keywords have the characteristic that they are reserved, but only in certain contexts. In the case of `override`, it has a reserved meaning only when it occurs at the end of a member function declaration. That means that if you have legacy code that already uses the name `override`, you don't need to change it for C++11:

```
class Warning {  
public:  
    // potential legacy class from C++98  
    ...  
    void override(); // legal in both C++98 and C++11  
    ...              // (with the same meaning)  
};
```

Other use of using `final` is that `final` prevents a class from being inherited.

```

class Base final {
public:
    void doWork() {}
};

class Derived : public Base { // ERROR: Invalid: Cannot i
nherit from a final class
public:
    void doSomething() {}
};

```

Reference-Qualified Member Functions (& / &&)

They restrict member functions based on whether `*this` is an lvalue or rvalue.

```

class Widget {
public:
    using Data = std::vector<double>;

    Data& data() & { return values; }           // lvalue →
return lvalue ref
    Data data() && { return std::move(values); } // rvalu
e → return by value (move)
};

auto vals1 = w.data();           // copies (OK)
auto vals2 = makeWidget().data(); // moves (efficient)

```

▼ Prefer `const_iterator` to iterators

What is a `const_iterator`?

- Similar to a **pointer-to-const**
- Points to values that **cannot be modified**
- Use it when you **don't need to modify elements**

- General rule: Use `const_iterator` whenever possible (just like `const` variables)

Problem in C++98 (Poor Support)

Using `iterator` was easy:

```
std::vector<int> values;

auto it = std::find(values.begin(), values.end(), 1983);
values.insert(it, 1998);
```

Trying to use `const_iterator` (C++98)

```
typedef std::vector<int>::iterator IterT;
typedef std::vector<int>::const_iterator ConstIterT; // typedefs

std::vector<int> values;
...
ConstIterT ci = std::find(static_cast<ConstIterT>(values.begin()),
                           static_cast<ConstIterT>(values.end()),
                           1983); // cast

values.insert(static_cast<IterT>(ci), 1998); // may not compile
```

In C++98, working with `const_iterator` was inconvenient and often impractical. Although `typedef` was used to simplify long iterator types (since `using` wasn't available yet), obtaining a `const_iterator` from a non-const container required awkward casts or indirect workarounds like binding the container to a const reference. Even after getting a `const_iterator`, things became more complicated because functions like

`insert` and `erase` only accepted regular `iterator`, not `const_iterator`, forcing developers to attempt conversions back—which were not portable and often didn't compile. In fact, there is no reliable way (not even with `static_cast` or `reinterpret_cast`) to convert a `const_iterator` to an `iterator`. As a result, despite the conceptual benefits, `const_iterator` was rarely used in C++98 due to its poor usability.

C++11 introduces:

- `cbegin()` and `cend()` → always return `const_iterator`
- `insert()` and similar functions accept `const_iterator`

```
// C++11 version of the above code example
std::vector<int> values;

auto it = std::find(values.cbegin(), values.cend(), 1983);
values.insert(it, 1998);
```

Generic Code with `cbegin` / `cend` :

For writing **generic code**, prefer non-member functions:

```
template<typename C, typename V>
void findAndInsert(C& container,
                  const V& targetVal,
                  const V& insertVal)
{
    using std::cbegin;
    using std::cend;

    auto it = std::find(
        cbegin(container),
        cend(container),
        targetVal
    );
}
```

```
    container.insert(it, insertVal);  
}
```

C++11 Limitation (Fixed in C++14)

- C++11 provides:
 - `begin`, `end` (non-member)
- Missing:
 - `cbegin`, `cend`, `cbegin`, `crend`

Fixed in **C++14**

Workaround in C++11

You can define your own `cbegin`:

```
template <class C>  
auto cbegin(const C& container) -> decltype(std::begin(container))  
{  
    return std::begin(container);  
}
```

Why this works

- `container` is `const`
- `std::begin(const container)` returns a **const_iterator**
- Works for:
 - STL containers
 - Built-in arrays
 - Custom containers

```
// Example with Arrays
```

```
int arr[] = {1, 2, 3};
```

```
auto it = std::cbegin(arr); // pointer-to-const (acts as c  
const_iterator)
```



1. Use `const_iterator` by default, unless you need to modify elements
2. In maximally generic code, prefer non-member versions of `begin`, `end`, `rbegin`, etc., over their member function counterparts.

▼ Declare functions `noexcept` if they won't emit exceptions

In C++98, exception specifications like `throw()` were fragile and hard to maintain, because any change in a function's implementation could require updating the specification, potentially breaking client code. C++11 replaced this with a simpler, more meaningful model based on `noexcept`, which states whether a function may emit exceptions or guarantees it won't. Declaring a function `noexcept` is important for both correctness and performance: compilers can generate significantly more optimized code because they don't have to preserve unwind-safe states. For example:

```
int f(int x) throw();    // C++98  
int f(int x) noexcept;  // C++11, more optimizable
```

Many STL operations—especially `std::vector` growth, move operations, and `swap`—depend on move constructors being `noexcept` to safely replace copies with moves:

```

struct Widget {
    Widget(Widget&&) noexcept;           // enables vector op
timizations
    Widget& operator=(Widget&&) noexcept;
};

```

Standard Library functions often use conditional `noexcept`:

```

template <class T, size_t N>
void swap(T (&a)[N], T (&b)[N])
    noexcept(noexcept(swap(*a, *b)));

```

This means higher-level swaps are `noexcept` only if lower-level swaps are. However, `noexcept` must be used carefully: once part of the interface, removing it can break callers, and forcing an implementation to be `noexcept` can lead to worse code if you contort logic just to avoid exceptions. Most functions should remain exception-neutral (they don't throw, but may let others' exceptions pass through) and therefore **not** be `noexcept`. But move operations, `swap`, destructors, and memory deallocation functions naturally should be `noexcept` and often are. Always declare a function `noexcept` only if you can commit that it will never emit exceptions.



1. In C++11, marking a function `noexcept` gives compilers stronger optimization opportunities than the old C++98 `throw()` specification. Both forms promise that a function won't emit exceptions, but when that promise is violated, `throw()` requires the runtime to fully unwind the stack before terminating, while `noexcept` allows the program to terminate *without guaranteeing full stack unwinding*. This seemingly small difference matters: because a `noexcept` function never needs to support stack unwinding, the compiler can generate faster code—it doesn't need to maintain unwindable stack state or emit cleanup logic to destroy objects in precise reverse order. By contrast, functions declared with `throw()` or with no exception specification must preserve unwindability, which limits optimization. In short:

```
RetType func() noexcept; // most optimizable
RetType func() throw();  // less optimizable
RetType func();           // less optimizable
```

2. Most functions in C++ are **exception-neutral**, meaning they don't throw exceptions themselves but may call other functions that do; when that happens, they simply let the exception propagate up the call stack to be handled elsewhere. Because these functions can still indirectly emit exceptions, they must **not** be marked `noexcept`. As a result, even though `noexcept` is useful, the majority of functions should not use it, since they correctly allow exceptions to pass through rather than guaranteeing that none will occur.

```
void doWork() { // exception-neutral
    setup();    // may throw
    process();  // may throw
    cleanup();  // may throw
    // exceptions propagate upward
}
```

3. Functions can be categorized by **contracts**: **wide contract** functions have no preconditions and can be safely called in any state (no undefined behavior), while **narrow contract** functions require certain preconditions to be met, and violating them leads to undefined behavior. Declaring a function `noexcept` is straightforward for wide-contract functions that are guaranteed not to throw, but for narrow-contract functions it becomes tricky—because even if their normal implementation doesn't throw, adding runtime checks for preconditions (e.g., for debugging) may require throwing exceptions. If such a function is marked `noexcept`, any thrown exception would terminate the program, making error handling impossible. Therefore, in practice, `noexcept` is best reserved for wide-contract functions where no exceptions are expected under any circumstances.

```
// Narrow contract example
void f(const std::string& s) noexcept { // pre
    condition: s.length() <= 32
    if (s.length() > 32) {
        throw std::runtime_error("Too long");
    // will call std::terminate
    }
}
```

▼ Use constexpr whenever possible

`constexpr` in C++11 indicates values that are **constant and (potentially) known at compile time**, but its behavior differs for objects and functions. **constexpr objects** are always `const` and must be initialized with compile-time values, making them usable in contexts requiring constant expressions (e.g., array sizes, template arguments). However, **constexpr functions are more flexible**: they can produce compile-time results *if called with compile-time constants*,

but behave like normal functions and execute at runtime otherwise. This allows a single function to serve both compile-time and runtime use cases. Compared to `const`, `constexpr` provides a stronger guarantee—**all `constexpr` are `const`, but not all `const` are `constexpr`.**

```
// constexpr object
constexpr int size = 10;
std::array<int, size> arr; // OK

// const is not enough
int x = 10;
const int size2 = x;
std::array<int, size2> arr2; // ERROR: (not compile-time
                             constant)
```

```
// constexpr function (works at compile-time and runtime)
constexpr int pow(int base, int exp) noexcept {
    return (exp == 0 ? 1 : base * pow(base, exp - 1)); //
    C++11 style
}

constexpr int val = pow(2, 3); // computed at compile-time

int a = 2, b = 3;
int result = pow(a, b); // computed at runtime
```

`constexpr` functions enable compile-time computation, improving performance and allowing usage in restricted contexts (like template arguments):

```
constexpr int n = 5;
std::array<int, pow(3, n)> data; // size computed at compile-time
```

They can also work with user-defined types if constructors and methods are `constexpr`:

```

class Point {
public:
    constexpr Point(double x, double y) noexcept : x_(x),
    y_(y) {}
    constexpr double x() const noexcept { return x_; }
    constexpr double y() const noexcept { return y_; }
private:
    double x_, y_;
};

constexpr Point p1(1.0, 2.0);

```



- `constexpr` objects → always compile-time constants
- `constexpr` functions → compile-time *when possible*, runtime otherwise
- Enables more optimization and broader usage contexts
- Stronger guarantee than `const`
- Part of the interface → removing it later can break client code

▼ Make const member functions thread safe